Learning-Based Vision-Guided Trajectory Generation for Autonomous Quadrotor Waypoint and Gate Navigation: Replacing Minimum-Snap Optimisation with a Deep Learning Pipeline

Project Report

Name: Aryan Amit Tiwari

Registration No.: 23BAI1409

1. Reference IEEE Paper

Title: Minimum Snap Trajectory Generation and Control for Quadrotors

Authors: D. Mellinger and V. Kumar

Published In: Proceedings of the 2011 IEEE International Conference on Robotics and Automation (ICRA), Shanghai, China, pp. 2520–2525, 2011 (IEEE). The paper received the Best Conference Paper Award at ICRA 2011.

DOI: 10.1109/ICRA.2011.5980409

Link: https://doi.org/10.1109/ICRA.2011.5980409

2. What Is This Paper About?

This paper addresses a core problem in autonomous quadrotor flight: how a small drone can fly smoothly and accurately through a prescribed sequence of 3-D positions (and yaw angles) in a tightly constrained indoor environment. Instead of flying straight lines between waypoints and stopping at each one, the authors generate a single smooth trajectory through all of them. They exploit the fact that a quadrotor is a *differentially flat* system: its full state and control inputs can be written as functions of just four flat outputs (x, y, z and yaw) and their derivatives. The trajectory is represented as piecewise polynomials and is found by minimising the integral of the squared *snap* (the fourth derivative of position), which is directly related to the changes in motor commands. This is cast as a quadratic program (QP) with waypoint and continuity constraints. A nonlinear controller is then designed to track the generated trajectory and the whole approach is validated on a real quadrotor.

3. Why Is Trajectory Generation for Quadrotors Difficult?

- Underactuated, fast dynamics: a quadrotor has six degrees of freedom but only four inputs, and it is unstable without feedback, so trajectories must be dynamically feasible, not just geometrically smooth.

- High-order smoothness requirements: motor thrusts relate to the fourth derivative of position, so trajectories need continuous derivatives up to snap. Low-order paths (straight lines, simple splines) cause aggressive, hard-to-track commands.
- Heavy mathematical machinery: the formulation needs differential flatness, high-order piecewise polynomial parametrisation, constrained QP solving, and an SE(3) nonlinear controller, which makes the method hard to re-implement, tune and extend.
- Time allocation: how much time to spend on each segment strongly changes the quality of the trajectory, and choosing good segment times is itself an optimisation problem.
- Re-planning cost: the optimisation must be re-run whenever waypoints change, which limits how quickly a vehicle can react when the waypoints come from a perception system that updates every frame.

4. Main Contributions of the Paper

- Shows that quadrotor dynamics are differentially flat, so trajectory planning can be done in the low-dimensional space of position and yaw rather than in the full 12-D state space.
- Formulates trajectory generation as minimisation of the integral of squared snap over piecewise polynomial segments, solved as a quadratic program subject to waypoint and derivative-continuity constraints, and runnable in real time.
- Handles both position and yaw trajectories through a sequence of 3-D waypoints in constrained spaces.
- Designs a nonlinear controller that tracks the generated trajectory accurately.
- Validates the full planning-and-control pipeline experimentally on a quadrotor.

5. How the Method Works (Methodology)

The method first uses differential flatness to express the quadrotor states and inputs through the flat outputs and their derivatives. A trajectory is then parametrised as a piecewise polynomial in each flat output, with one polynomial per segment between consecutive waypoints. The authors minimise a cost that penalises the squared snap integrated over time. Because the cost is quadratic in the polynomial coefficients and all the constraints (passing through waypoints, matching derivatives at segment boundaries, and fixed start/end conditions) are linear, the problem is a QP that can be solved efficiently. Segment durations are supplied to the optimiser rather than learned from data, so they have to be chosen by the user or by a heuristic. Finally, a nonlinear controller computes the desired thrust and body moments from the position, velocity and attitude errors with respect to the reference trajectory, closing the loop on the real vehicle.

6. Real-World Applications Mentioned

- Indoor flight of small quadrotors through cluttered, tightly constrained spaces.
- Aerial inspection and mapping where smooth, energy-aware trajectories through chosen waypoints are needed.
- Aggressive manoeuvres such as flying through gaps, hoops or narrow openings at speed.

- Autonomous navigation tasks that need a reliable bridge between a high-level waypoint plan and low-level motor control.

7. Overall Takeaway

The paper is significant because it turned quadrotor trajectory planning into a clean, computable problem by combining differential flatness with minimum-snap optimisation, and it remains the standard baseline that most later planners are compared against. At the same time, its strength is also its practical limit for a student project: the pipeline assumes that waypoints are given, requires careful mathematics to implement, and needs an optimisation solve each time the waypoints change. For a project building on this paper, the key ideas to carry forward are: (1) smooth, dynamically feasible trajectories through a sequence of waypoints are the right interface between perception and control, (2) smoothness (low snap) is the quality measure that matters, and (3) the optimisation behind it is a mapping from waypoints to a trajectory, which is exactly the kind of mapping a neural network can learn.

8. Proposed Project

Project Title: Learning-Based Vision-Guided Trajectory Generation for Autonomous Quadrotor Waypoint and Gate Navigation: Replacing Minimum-Snap Optimisation with a Deep Learning Pipeline

Base Reference: D. Mellinger and V. Kumar, Proceedings of the 2011 IEEE International Conference on Robotics and Automation (ICRA) (see Sections 1–7 above)

Implementation Platform: Python, PyTorch, OpenCV, Ultralytics YOLO, gym-pybullet-drones (PyBullet physics simulator) [3][4]

8.1 Objective

To design and simulate a software pipeline in which a quadrotor sees a sequence of gates (or target markers) through its onboard camera, converts the detections into 3-D waypoints, and uses a trained neural network to generate a smooth, dynamically feasible trajectory through them in a single forward pass, instead of solving the minimum-snap optimisation at run time. The project then evaluates the learned planner against the classical minimum-snap QP and a naive straight-line planner in terms of trajectory quality, tracking accuracy, mission success and computation time.

8.2 Project Scope

Rather than re-deriving the differential-flatness analysis, the piecewise-polynomial QP and the SE(3) nonlinear controller of the reference paper, this project scopes the problem down to a data-driven, software-only pipeline. A camera-based detector supplies the waypoints, a neural network learns the waypoint-to-trajectory mapping, and the existing PID flight controller of the simulator (optionally with a small learned residual) tracks the output. The minimum-snap QP is used only *offline* as a teacher that produces training labels and as a baseline for comparison, so the project does not need the full mathematical derivation to run. This keeps the work achievable within a semester while preserving the paper's central idea: smooth, low-snap trajectories through waypoints are the correct interface between perception and control. Adding a modern vision front-end and a learned planner is the new contribution.

8.3 System Model

The UAV is a simulated quadrotor (the Crazyflie 2.x-style model in gym-pybullet-drones) flying in a PyBullet world with a randomised course of several gates placed at varying positions, heights and orientations. A forward-facing RGB camera renders frames at a fixed rate. The states used by the planner are the drone's position, velocity and yaw (from simulator odometry) together with the estimated gate positions in the camera/body frame. Low-level motor control and rigid-body dynamics are handled by the simulator, so the project effort can concentrate on perception and planning.

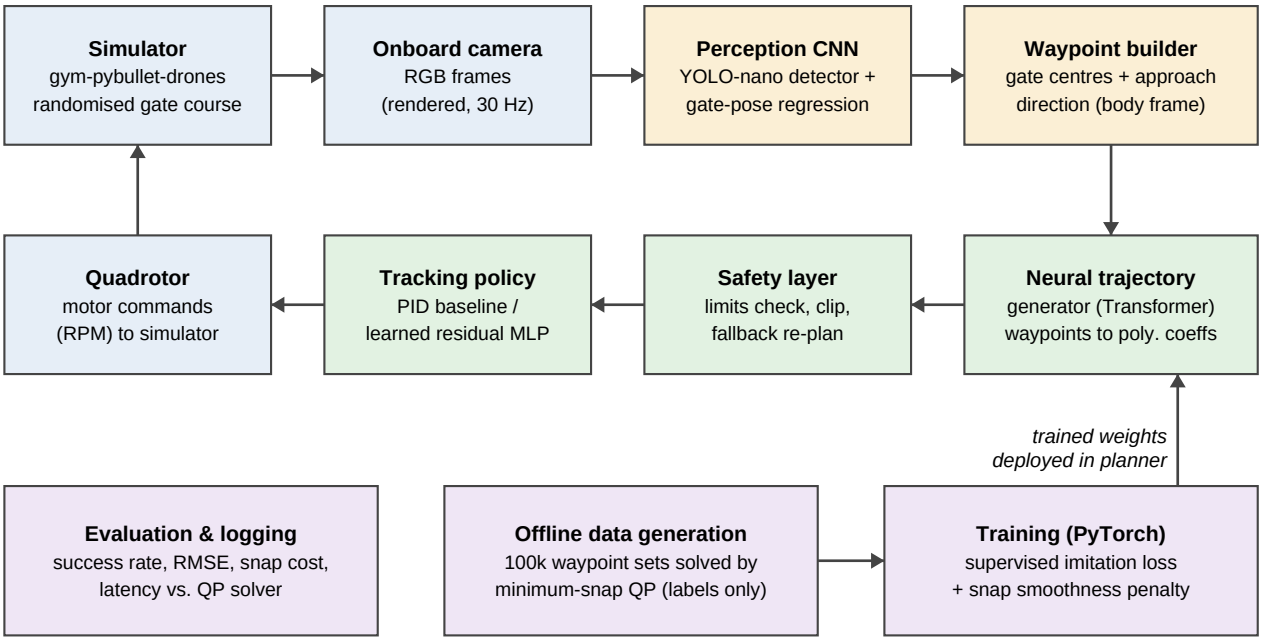

Figure 1: Proposed system architecture (online flight loop and offline training/evaluation loop).

8.4 Perception and Planning Pipeline

- Gate / waypoint detection: a lightweight YOLO-family detector finds gates (or coloured markers) and their bounding boxes in each frame. Training images are generated automatically from the simulator with domain randomisation of lighting, background and texture.
- Gate pose estimation: a small CNN head (or a PnP step using the known gate size as a fallback) regresses the relative position, distance and orientation of the next gate from the bounding-box crop, giving a 3-D waypoint and an approach direction in the body frame.
- Neural trajectory generator: a compact Transformer encoder takes the ordered list of waypoints (position and yaw, plus the current drone state) and outputs the segment durations and polynomial coefficients for the trajectory. It is trained by supervised imitation of the minimum-snap solution on about 100,000 randomly generated waypoint sets, with an added snap-based smoothness loss so the output stays smooth even when it deviates from the label. This replaces the run-time QP and the hand-chosen time allocation. A related study already shows that Transformers can predict time allocations for a minimum-snap generator [2].

- Trajectory tracking: the simulator's PID controller tracks the generated reference. As a stretch goal, a small MLP policy trained by imitation learns a residual correction on top of the PID command, standing in for the paper's nonlinear controller.
- Safety layer: velocity, acceleration and tilt limits are checked on every generated trajectory. If a limit is violated or a gate detection is lost, the layer slows the trajectory or re-plans from the last good waypoints.
- Moving or updated targets: whenever the detector updates a gate position, the trajectory is regenerated by a single forward pass, which is fast enough to do at each perception update.

8.5 System Architecture

- Simulation environment: gym-pybullet-drones with a procedurally generated gate course (different layouts, gate sizes and heights), providing quadrotor dynamics and camera rendering.
- Perception module: runs the detector and pose-regression network on each camera frame and publishes the estimated gate waypoints.
- Planning module: the neural trajectory generator plus safety layer; outputs a time-parameterised reference trajectory to the tracker.
- Baseline planners: (a) the classical minimum-snap QP using the same waypoints and (b) a straight-line waypoint follower, both run under identical conditions for comparison.
- Data and training module: dataset generator (offline minimum-snap solutions as labels), PyTorch training scripts, and experiment configuration files.
- Logging module: records detection accuracy, trajectory snap cost, tracking error, flight time, mission outcome and per-stage latency for later analysis.

8.6 Datasets and Training Plan

Component	Data	Training approach
Gate detector	About 10,000 simulator frames with automatically generated boxes, domain-randomised	Fine-tune a pretrained YOLO-nano model
Gate pose network	Same frames with ground-truth gate pose from the simulator	Supervised regression, L1 loss on position and angle
Trajectory generator	About 100,000 random waypoint sets with minimum-snap solutions as labels	Supervised imitation + snap smoothness penalty; compared against an MLP baseline
Residual tracker (stretch)	Rollouts of PID tracking in simulation	Behaviour cloning of the correction term

8.7 Evaluation Metrics

- Detection quality: mAP, and precision/recall of gate detection on held-out simulated frames.
- Pose-estimation error: mean position error (metres) and heading error (degrees) of the estimated gate pose.

- Trajectory quality: integral of squared snap of the learned trajectory relative to the minimum-snap optimum, and waypoint-passing error.
- Tracking performance: RMSE of the executed path relative to the reference trajectory.
- Mission success rate: fraction of randomly generated gate courses completed without a missed gate or crash, for the learned planner and both baselines.
- Computation time: planner inference time versus QP solve time, and end-to-end perception-to-command latency, to assess real-time feasibility.
- Ablations: Transformer versus MLP planner, with and without the smoothness loss, with and without domain randomisation.

8.8 Project Timeline (14 Weeks)

Weeks	Work package	Deliverable
1–2	Literature review, simulator setup, baseline PID flight	Drone flies a fixed waypoint list in simulation
3–4	Minimum-snap baseline and dataset generator	100k labelled waypoint-trajectory pairs
5–7	Neural trajectory generator: model, training, ablations	Planner reproducing QP trajectories
8–10	Gate detector and pose network with simulator data	Detector + pose estimates from camera
11–12	Integration, safety layer, closed-loop gate-course flights	End-to-end vision-to-flight demo
13	Full evaluation against baselines, stretch residual tracker	Result tables and plots
14	Final report and demo video	Submission

8.9 Expected Outcomes and Risks

- Expected outcome: the learned planner should produce trajectories close to the minimum-snap optimum (measured by snap ratio and waypoint error) at a much lower per-call cost, which is what allows re-planning at every perception update. These are targets to be verified by experiments, not results already obtained.
- Risk – the network produces a trajectory that violates limits: mitigated by the smoothness loss and the safety layer with fallback re-planning.
- Risk – simulation-to-real gap in perception: mitigated by domain randomisation; real flights are outside this semester's scope.
- Risk – time overrun: the learned tracker is a stretch goal, and the core deliverable is the vision-to-learned-planner pipeline with a PID tracker.

8.10 Future Extension

As identified above, replacing the PID tracker with a fully learned controller, or with reinforcement learning (as used in learning-based high-speed flight [5]), would move the project closer to a complete replacement of the reference paper's control law. Adding obstacle-aware planning, e.g. by feeding depth or detection information to the generator, and deploying the pipeline on a real quadrotor with an onboard companion computer (such as a Jetson-class board) are proposed as further work beyond the current semester's scope.

9. Conclusion

This project uses D. Mellinger and V. Kumar's 2011 IEEE ICRA paper on minimum-snap trajectory generation and control for quadrotors as its foundational reference. The proposed pipeline keeps the paper's core motivation, namely smooth, dynamically feasible trajectories through waypoints as the interface between perception and control, but replaces the hand-derived mathematics (differential flatness, polynomial QP and nonlinear controller) with a camera-based detector and a neural trajectory generator trained on QP-generated data. The result is a modern, learning-based, simulation-only project suitable for a semester, with a clear and measurable comparison to the classical method and a defined path to a fully learned controller and real-drone deployment as future work.

References

[1] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in Proc. 2011 IEEE International Conference on Robotics and Automation (ICRA), Shanghai, China, 2011, pp. 2520–2525. DOI: 10.1109/ICRA.2011.5980409.

[2] S. Tankasala and M. Pryor, “Accelerating Trajectory Generation for Quadrotors Using Transformers,” Learning for Dynamics and Control Conference (L4DC), 2023. arXiv:2303.15606.

[3] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, “Learning to Fly – a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control,” in Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2021.

[4] G. Jocher et al., Ultralytics YOLO (open-source object-detection framework), https://github.com/ultralytics/ultralytics.

[5] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, “Learning high-speed flight in the wild,” Science Robotics, vol. 6, no. 59, 2021. DOI: 10.1126/scirobotics.abg5810.